SEO Optimization Planner

First-order logic + search over Advanced Web Ranking data

Topic 5 · Project report

A planner that reads ranking data, reasons about it with logic, and searches for the best to-do list.

Topic 5: Integrating Logic and Search: A Simple Planner. Sabina Comandasu, October 2026.

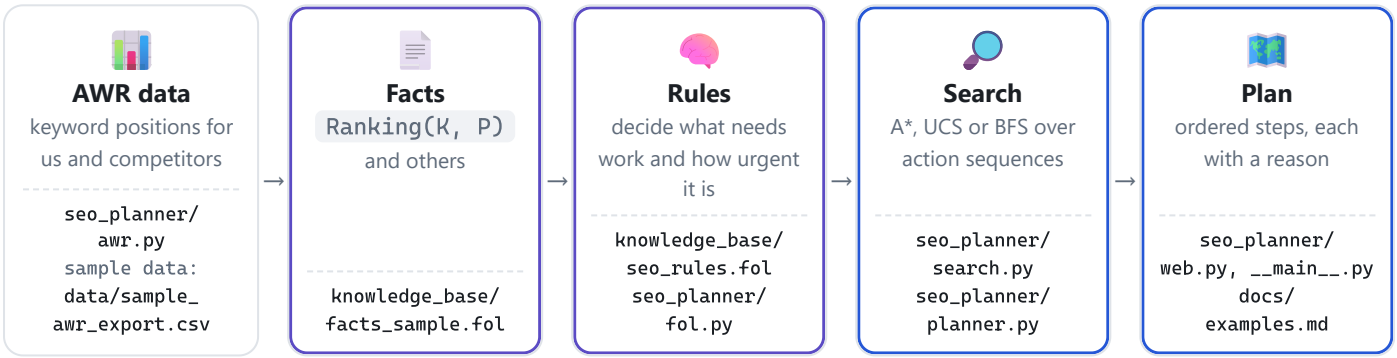
33
FOL rules in the knowledge base

10
planning actions (STRIPS style)

3
search algorithms: BFS, UCS, A*

0
external libraries (pure Python)

HOW IT WORKS, END TO END (AND WHICH FILES TO OPEN FOR EACH STAGE)



In plain words. An SEO team tracks where their website appears on Google for many keywords. Every week some keywords slip, and competitors overtake others. Deciding what to fix first is usually guesswork. This planner turns that decision into logic (rules about what "needs work" means) and search (finding the cheapest sequence of tasks that handles every important keyword).

THE ASSIGNMENT, AND WHERE EACH PART LIVES

Requirement	How the project meets it	Code / file
Define the problem domain	SEO optimisation: decide which tracked keywords need work, and which actions to take in which order. Section 1.	`planner.py`, `awr.py`
Represent rules in FOL	33 Horn-clause rules with negation-as-failure, run by a hand-written inference engine with unification. Section 2.	`seo_rules.fol`, `fol.py`
Use search to find a valid sequence of actions	BFS, uniform-cost search and A* with a domain heuristic. Action preconditions are FOL queries. Sections 3–4.	`search.py`, `planner.py`
Deliverable: planner code	Python package with a command line and a local web app, plus unit tests.	`seo_planner/`, `tests/`
Deliverable: example plans	Real program output on the sample data. Section 5.	`docs/examples.md`
Deliverable: documentation	This report, the README, and the in-app documentation page.	`README.md`, `docs.html`

1 The problem domain

Search engine optimisation (SEO) for a website whose rankings are tracked in Advanced Web Ranking (AWR).

THE WORLD

A website tracks a list of **keywords**. For each one, AWR records the website's **position** on Google (#1 is the top result) at each ranking update, along with the positions of **competitor** websites. Keywords are organised into **keyword groups**.

THE QUESTION THE PLANNER ANSWERS

"Given today's rankings, which keywords need attention, how urgent is each one, and what is the cheapest ordered list of SEO tasks that addresses all of them?"

STATE

What work has been done so far, as a set of facts such as `Audited(K)` or `ContentImproved(K)`. The plan starts with nothing done.

ACTIONS

10 SEO tasks (audit a page, analyse competitors, rewrite content…). Each has preconditions, effects and an effort cost.

GOAL

Every keyword in scope has been fully addressed and a ranking re-check has been scheduled for it: `∀K (InScope(K) → Rechecked(K))`.

Example data used in this report

A fictional shop, **shoeshop.example**, competing with **runfast.example** and **sportzone.example** on Google USA. The current update (19 Oct) is compared with the previous one (12 Oct). The same pipeline runs on a live AWR account through the API.

Keyword	Group	Now	Before	Change	Best competitor	Rules conclude…
trail running shoes	Running Shoes	#14	#5	▼ 9	runfast #6	Priority 1 · dropped
womens running shoes	Running Shoes	#9	#2	▼ 7	sportzone #4	Priority 1 · dropped
best running shoes	Running Shoes	#12	#11	▼ 1	runfast #4	Priority 2 · quick win
marathon shoes	Running Shoes	#17	#18	▲ 1	runfast #25	Priority 2 · quick win
sneaker cleaning kit	Accessories, Sneakers	#19	#18	▼ 1	sportzone #6	Priority 2 · quick win
waterproof sneakers	Sneakers	#7	#6	▼ 1	runfast #3	Priority 2 · quick win
running shoes online	Running Shoes	#25	#24	▼ 1	runfast #8	Priority 3
running shoes sale	Deals	#33	#35	▲ 2	none	Priority 3
cheap running shoes	Deals, Running Shoes	not ranking	not ranking	·	runfast #12	Priority 3 · no page
kids running shoes	Running Shoes	#2	#3	▲ 1	runfast #5	Healthy
running socks	Accessories	#8	#9	▲ 1	sportzone #12	Healthy

The last column is not stored anywhere. The FOL rules in Section 2 derive it from the raw positions.

2 Knowledge representation in first-order logic

The domain knowledge is a set of logic rules, kept separate from the code in `knowledge_base/seo_rules.fol`.

Facts: what AWR tells us

Predicate	Meaning
`Tracked(K)`	Keyword K is tracked in the AWR project
`Ranking(K, P)`	Our current position for K (101 means not in the top 100)
`PreviousRanking(K, P0)`	Our position at the earlier update we compare with
`CompetitorRanking(K, C, P)`	Competitor C's best position for K
`RankingUrl(K, U)`	Our page U that ranks for K
`InGroup(K, G)`	K belongs to AWR keyword group G

Rules: what the facts mean

Every rule reads `Head :- Body`, meaning *"for all values of the variables, if the body is true then the head is true"*. `not` means *"cannot be proven"* (negation as failure, under a closed-world assumption). A selection of the 33 rules:

CLASSIFYING KEYWORDS

"Striking distance": we rank #4–#20, close enough that a little work can move it up.

```
StrikingDistance(K) :- Ranking(K, P), P >= 4, P <= 20.
```

A keyword dropped if it fell 5 or more positions since the previous update.

```
Dropped(K) :- Ranking(K, P), PreviousRanking(K, P0), D is P - P0, D >= 5.
```

Competitor C beats us on a keyword if C ranks higher (a smaller number) than we do.

```
BeatenBy(K, C) :- Ranking(K, P), CompetitorRanking(K, C, Q), Q < P.
```

DECIDING WHAT NEEDS WORK, AND HOW URGENTLY

Anything outside page 1 (below #10) needs work.

```
NeedsOptimization(K) :- Ranking(K, P), P > 10.
```

A keyword close to the top that a competitor is beating needs work.

```
NeedsOptimization(K) :- StrikingDistance(K), LosingToCompetitor(K).
```

A tracked keyword is healthy when none of the "needs work" rules apply.

```
Healthy(K) :- Tracked(K), not NeedsOptimization(K).
```

Urgent keywords (needs work and dropped) get priority 1; quick wins in striking distance get priority 2.

```
Priority(K, 1) :- Urgent(K).
Priority(K, 2) :- NeedsOptimization(K), not Urgent(K), StrikingDistance(K).
```

RULES ABOUT THE PLAN'S PROGRESS

A keyword is fully addressed when its page is improved, internally linked, and any drop and competitor gap are handled.

```
Addressed(K) :- Improved(K), InternalLinksAdded(K), DropHandled(K), GapHandled(K).
```

Don't start a keyword while a more urgent keyword in the plan is still unfinished.

```
Blocked(K) :- HigherPriority(J, K), InScope(J), not Addressed(J).
```

The inference engine (`seo_planner/fol.py`)

UNIFICATION

Matches a pattern such as `Ranking(K, P)` against facts and binds the variables (`K = 'marathon shoes'` , `P = 17`). Includes the occurs check.

FORWARD CHAINING

Applies all rules repeatedly until no new facts appear. Rules are sorted into **strata** (layers) so that a `not X` is only checked after X is completely known.

QUERIES

Any conjunction can be asked, for example `FirstPage(K), BeatenBy(K, C)` returns every keyword on page 1 that a competitor is beating.

PROOFS

Every derived fact records which rule produced it, so the program can always answer *"why is this true?"*

Example: why is "trail running shoes" priority 1?

- **Priority('trail running shoes', 1)** because it is urgent
  - Urgent('trail running shoes') needs work AND dropped
    - NeedsOptimization('trail running shoes') outside page 1
      - Ranking('trail running shoes', 14) AWR data
      - 14 > 10 calculation
    - Dropped('trail running shoes') fell 5+ positions
      - Ranking('trail running shoes', 14) AWR data
      - PreviousRanking('trail running shoes', 5) AWR data
      - 9 is 14 − 5, and 9 ≥ 5 calculation

More example queries (real output)

WHICH KEYWORDS DROPPED, AND BY HOW MUCH?

`Dropped(K), Ranking(K, P), PreviousRanking(K, P0), D is P - P0`

```
'womens running shoes'  #2 → #9   D = 7
'trail running shoes'   #5 → #14  D = 9
```

ON PAGE 1 BUT STILL BEATEN?

`FirstPage(K), BeatenBy(K, C), CompetitorRanking(K, C, Q)`

```
'waterproof sneakers'  runfast.example   #3
'womens running shoes' runfast.example   #5
'womens running shoes' sportzone.example #4
```

How priority is set

Priority is **not set in Python code**. The rules in the "Prioritisation" section of `knowledge_base/seo_rules.fol` derive it from the AWR positions, in three questions:

STEP 1

Does the keyword need work?

Yes if it is **outside page 1** (below #10), it **dropped** 5+ places, or it ranks #4–#20 and **a competitor is ahead**. If none apply, it is Healthy and gets no priority.

```
NeedsOptimization(K) :-
  Ranking(K, P), P > 10.
NeedsOptimization(K) :-
  Dropped(K).
NeedsOptimization(K) :-
  StrikingDistance(K),
  LosingToCompetitor(K).
```

→

STEP 2

Is it urgent?

Urgent means it needs work **and** dropped. "Dropped" is a fall of at least 5 positions since the update you compare with.

```
Urgent(K) :- NeedsOptimization(K),
  Dropped(K).
Dropped(K) :- Ranking(K, P),
  PreviousRanking(K, P0),
  D is P - P0, D >= 5.
```

→

STEP 3

Which priority number?

Urgent keywords get 1. Otherwise, "striking distance" (#4–#20) gets 2, a quick win. Everything else gets 3.

```
Priority(K, 1) :- Urgent(K).
Priority(K, 2) :-
  NeedsOptimization(K),
  not Urgent(K),
  StrikingDistance(K).
Priority(K, 3) :-
  NeedsOptimization(K),
  not Urgent(K),
  not StrikingDistance(K).
```

Priority	Meaning	Keywords in the sample data
Priority 1	Needs work and dropped 5+ places	trail running shoes (#5 → #14), womens running shoes (#2 → #9)
Priority 2	Needs work, not urgent, ranks #4–#20 (a quick win)	best running shoes #12, marathon shoes #17, sneaker cleaning kit #19, waterproof sneakers #7
Priority 3	Needs work, not urgent, below #20 or not ranking	running shoes online #25, running shoes sale #33, cheap running shoes (not ranking)
Healthy	None of the "needs work" rules apply	kids running shoes #2, running socks #8

How the planner uses priority

1 · CHOOSING WHICH KEYWORDS TO PLAN

`_priority_key()` in `seo_planner/planner.py` sorts the keywords that need work by priority, then alphabetically. The planner takes the top N (5 by default, or the "how many keywords" setting in the app).

2 · ORDERING THE WORK

Two more rules make the planner finish more urgent keywords first. "Audit page" requires `not Blocked(K)`, so no priority-2 keyword is started until every priority-1 keyword is addressed. `--no-strict-priority` turns this off.

```
HigherPriority(J, K) :- Priority(J, A),
Priority(K, B), A < B.
Blocked(K) :- HigherPriority(J, K), InScope(J),
not Addressed(J).
```

Changing how priority works. Edit the numbers in the rules: for example, `D >= 5` → `D >= 3` in `Dropped` to treat smaller drops as urgent, or the 4 and 20 in `StrikingDistance`. No Python changes are needed.

Checking one keyword. In the app's **Query** tab, ask `Priority('trail running shoes', P)` with *Why is this true?*. On the command line, add `--why`. The proof tree on the previous page is exactly this.

3 Actions: where logic meets search

Each action is a STRIPS-style schema. Its **precondition is an FOL query**: the planner may take the action for a keyword only if the knowledge base proves the precondition for that keyword.

🔍 Research ✍️ Content 🔧 Technical 📈 Measure

	Action	Allowed when… (precondition, simplified)	Adds	Effort
	**Audit page**	The keyword is in the plan and not blocked by a more urgent one	`Audited`	1
	**Investigate drop**	Audited and `Dropped(K)`	`DropInvestigated`	2
	**Scout competitors**	Audited and `LosingToCompetitor(K)`	`CompetitorsAnalyzed`	2
	**On-page tune-up**	Audited and in striking distance (#4–#20)	`OnPageOptimized`	2
	**Close content gap**	Competitors analysed and we have a ranking page	`ContentImproved`	4
	**Rewrite content**	Audited, has a page, not in striking distance, and ranking	`ContentImproved`	5
	**Build landing page**	Audited and `NotRanking(K)` (below #50)	`HasLandingPage`	8
	**Supporting article**	Audited, has a page, content not yet improved	`ContentImproved`, `InternalLinksAdded`	7
	**Internal links**	`Improved(K)`	`InternalLinksAdded`	1
	**Re-check rankings**	`Addressed(K)`, the goal-defining rule	`Rechecked`	1

Why this is "logic + search". In each state the planner calls the inference engine to find every action whose precondition is provable, along with the variable bindings (which keyword). Search then decides *which* of those valid actions to take next. Logic says what is *allowed*, and search finds what is *best*.

Honest modelling. Actions only record that *work was done* (for example, content was improved). They never pretend to change rankings. Whether a ranking actually improves is measured by the next AWR update, which is why every plan ends with "Re-check rankings".

4 Search: finding the sequence of actions

The planner is a standard state-space search problem: start state, successor function, goal test and step costs. Three algorithms are implemented in `seo_planner/search.py`.

BFS

Fewest steps. Explores level by level and ignores effort. Finds the shortest plan, which is not always the cheapest.

UNIFORM-COST SEARCH

Cheapest plan. Always expands the state with the lowest effort so far. Optimal, but explores a lot.

A* (DEFAULT)

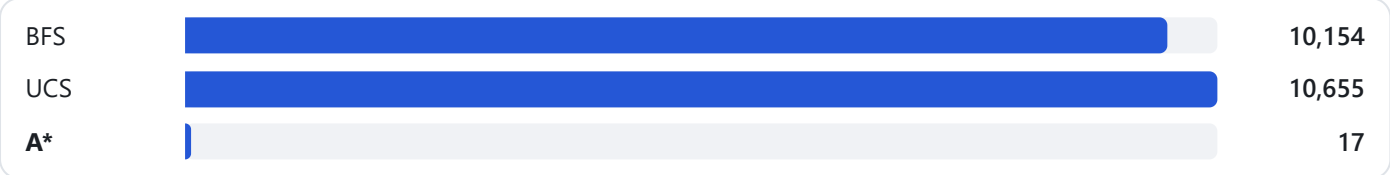
Cheapest plan, fast. Like UCS, plus a heuristic estimate of the remaining effort to steer it toward the goal.

The A* heuristic

For every keyword that is not finished yet, it adds up the cheapest cost of each step the rules make **unavoidable**. For example: a re-check (1), internal links (1), an audit if not done (1), a drop investigation if the keyword dropped (2), and a landing page if we are not ranking (8). Because it only counts steps that can't be skipped, it never overestimates (it is **admissible**), so A* still returns the cheapest plan.

How much work each algorithm does

Three keywords planned without the priority ordering, which gives the largest state space in the examples. Number of states expanded:



Algorithm	Plan steps	Total effort	States expanded	States generated	Time
BFS	14	37	10,154	62,838	17.12 s
UCS	17	**31**	10,655	64,450	18.06 s
A*	17	**31**	**17**	**97**	**0.12 s**

What this shows. UCS and A* both find the cheapest plan (effort 31). BFS finds a shorter plan (14 steps) that costs more (37). A* reaches the optimal answer after expanding only 17 states, about 600 times fewer than UCS, because the heuristic sends it straight down the right path.

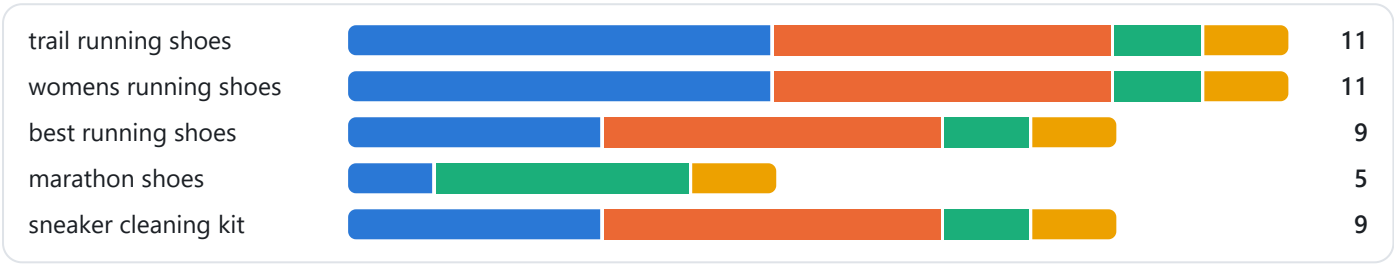
5 Example plans

Real output from `python -m seo_planner plan knowledge_base/facts_sample.fol`: the five most urgent keywords, A*, strict priority order.

5	26	45	26	0.26 s
keywords in scope	steps	total effort (optimal)	states expanded by A*	search time

EFFORT PER KEYWORD, BY TYPE OF WORK

Research Content Technical Measure



Notice the **priority rule at work**. `Blocked(K)` stops the planner from starting a priority-2 keyword until both priority-1 keywords are `Addressed`, so the urgent work (steps 1–10) always comes first.

THE PLAN AS QUESTS (NUMBERS SHOW THE ORDER CHOSEN BY THE SEARCH)

trail running shoes — Priority 1

Was #5, now #14 · recover the drop and get back on page 1 · 11

1	Audit the page and SERP	1
2	Investigate why it dropped (#5 → #14)	2
3	Scout runfast #6, sportzone #11	2
4	Close the content gap	4
5	Add internal links	1
25	Re-check rankings	1

womens running shoes — Priority 1

Was #2, now #9 · win back the top spots · 11

6	Audit the page and SERP	1
7	Investigate why it dropped (#2 → #9)	2
8	Scout sportzone #4, runfast #5	2
9	Close the content gap	4
10	Add internal links	1
26	Re-check rankings	1

best running shoes — Priority 2

Quick win: break into page 1 · 9

11	Audit the page (#12)	1
12	Scout runfast #4, sportzone #9	2
13	Close the content gap	4
14	Add internal links	1
22	Re-check rankings	1

marathon shoes — Priority 2

Quick win: no competitor ahead, just tune the page · 5

15	Audit the page (#17)	1
16	Optimise title, meta and headings	2
17	Add internal links	1
23	Re-check rankings	1

sneaker cleaning kit — Priority 2

Quick win: catch up with sportzone #6 · 9 · steps 18 Audit (#19) → 19 Scout competitors → 20 Close content gap → 21 Internal links → 24 Re-check

Example: fewest steps is not the cheapest plan

Keyword **running shoes sale** (#33, priority 3). BFS minimises the number of steps, while A* minimises total effort.

BFS

3 steps · 9

1	Audit the page	1
2	Publish a supporting article	7
3	Re-check rankings	1

A* ✓ cheaper

4 steps · 8

1	Audit the page	1
2	Rewrite and expand the content	5
3	Add internal links	1
4	Re-check rankings	1

Example: keyword with no page at all

cheap running shoes is not in the top 100 and has no ranking URL. The rules make “Build landing page” the only way to get a page, and the content work can only start after it exists.

1	Audit the page and SERP (not ranking, no ranking page)	1
2	Create a dedicated landing page	8
3	Analyse runfast #12 and sportzone #20	2
4	Add the topics competitors cover that our page misses	4
5	Add internal links	1
6	Re-check rankings	1

6 steps · 17 · A* expanded 6 states.

Every step comes with its reason

The planner prints the proven precondition behind each action, so any step can be checked by hand:

```
CloseContentGap('cheap running shoes')
  because CompetitorsAnalyzed('cheap running shoes'),
          RankingPageExists('cheap running shoes'),
          not ContentImproved('cheap running shoes')
```

6 Using the planner

WEB APP

```
python -m seo_planner.web
```

1. Paste an AWR API key, or click **Use sample data**.
2. Choose the project, the ranking date, the date to compare with, and a keyword group.
3. Choose the website, how many keywords to plan, and the algorithm.
4. Read the plan as 🗺️ **Quests**, a 📊 **Timeline**, or 📋 **Steps**.

COMMAND LINE

```
python -m seo_planner import
data/sample_awr_export.csv \
    --site shoeshop.example

python -m seo_planner query
knowledge_base/facts_sample.fol \
    "Priority(K, P)" --why

python -m seo_planner plan
knowledge_base/facts_sample.fol \
    --algorithm astar --compare
```

APP TABS

Keywords: the rules' classification · **Plan**: steps with a "why is this allowed?" proof, plus downloads · **Query**: ask FOL questions in plain English or logic · **Rules**: every rule with its plain-English description.

TESTS

Unit tests cover unification, the knowledge base, the AWR parsing and the planner. Plan validity is checked by **replaying** each plan: every action's precondition must hold when it runs, and the goal must hold at the end.

Project layout

File	What it does
`knowledge_base/seo_rules.fol`	The 33 rules: all the domain knowledge
`seo_planner/fol.py`	FOL terms, parser, unification, stratified forward chaining, queries, proofs
`seo_planner/planner.py`	Action schemas, the planning problem, the A* heuristic, explanations
`seo_planner/search.py`	BFS, uniform-cost search, A*
`seo_planner/awr.py`	AWR API v3 client and CSV import: turns rankings into facts
`seo_planner/__main__.py`, `web.py`	Command-line interface and local web app
`docs/examples.md`	Complete example queries and plans with real output
`tests/`	Unit tests

Deliverables checklist

- ✓ **Problem domain defined:** SEO optimisation of tracked keywords, using real AWR data
- ✓ **Rules represented in FOL:** 33 rules in `seo_rules.fol`, with a hand-written inference engine
- ✓ **Search finds a valid sequence of actions:** BFS, UCS and A*, with plans verified by replay
- ✓ **Planner code:** `seo_planner/`, pure Python with no dependencies
- ✓ **Examples of generated plans:** Section 5 of this report and `docs/examples.md`
- ✓ **Documentation:** this report, `README.md` and the in-app documentation page